Upskill by Building a Live AI Project (**Free Workshop** *with a Chance for Long-Term Mentorship*)

Development Doc

0. Intro from students

Name:
Year:
Interests: Any projects or particular framework anything?

1. Introduction to Hexmos

The session begins with a brief introduction to **Hexmos** and the workshop facilitators, with a simple demo of the end goal https://maneshwar.onrender.com/.

2. Workshop Introduction M

https://docs.google.com/presentation/d/1Ha4OEN-zSHuX_PI7BfZyX6FYhQVrbKFEcMq0Bo-deFo/edit?usp=sharing

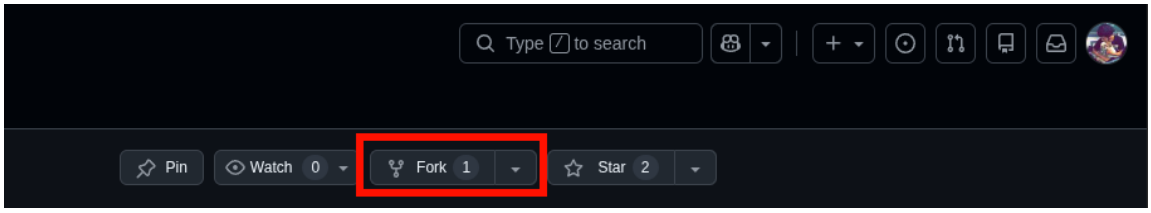
3. Industry Developer Workflow: GitHub & Version Control L

Topics covered:

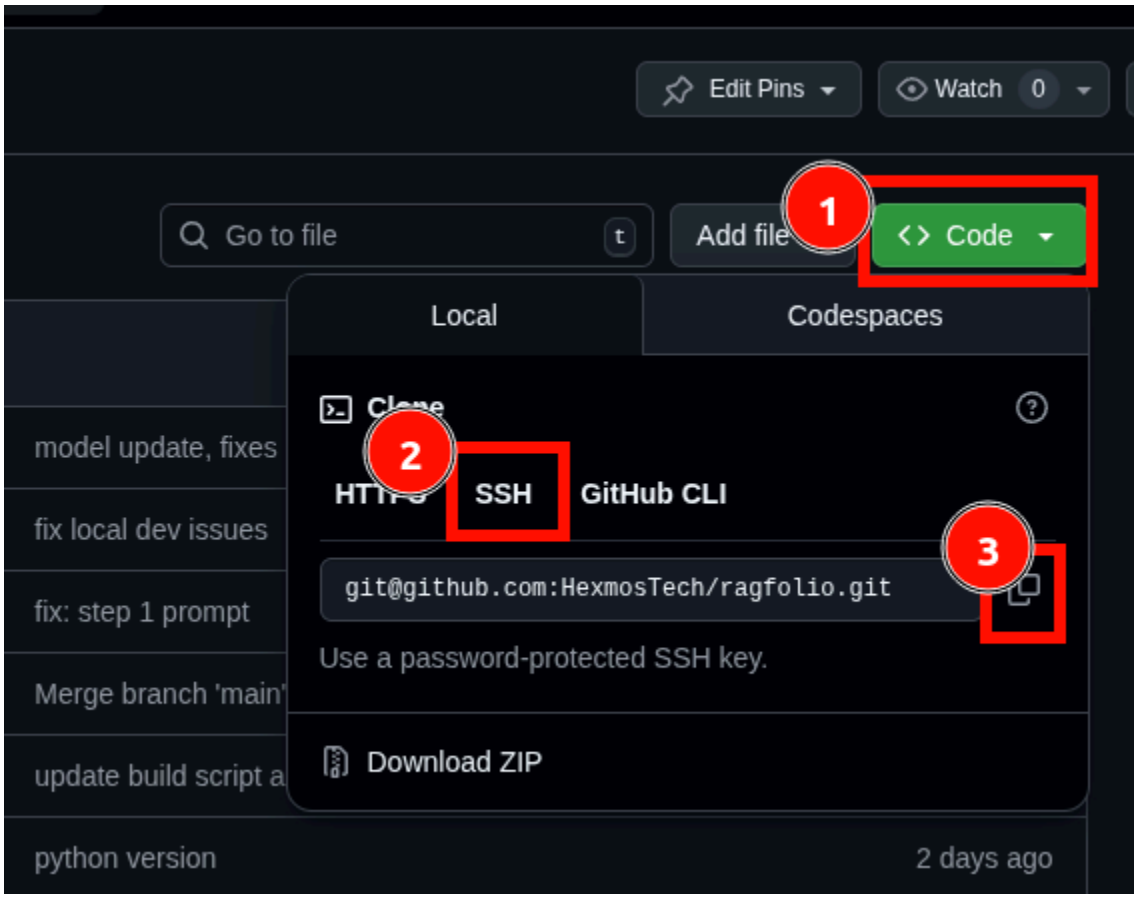
- What **GitHub** is
- What a **repository** is
- Why version control is important

Students will:

1. **Fork the Ragfolio repository template**



2. Clone the forked repository locally, not the original template repository.



3. Set up Git SSH configuration

4. Clone the repository

```
lovestaco@i3nux-mint:~$ z pers
lovestaco@i3nux-mint:~/pers$ git clone https://github.com/my_github_account_name/ragfolio.git
```

4. Understanding Core Web Development Concepts M

Before moving into AI systems, students are introduced to the basic architecture of a web application.

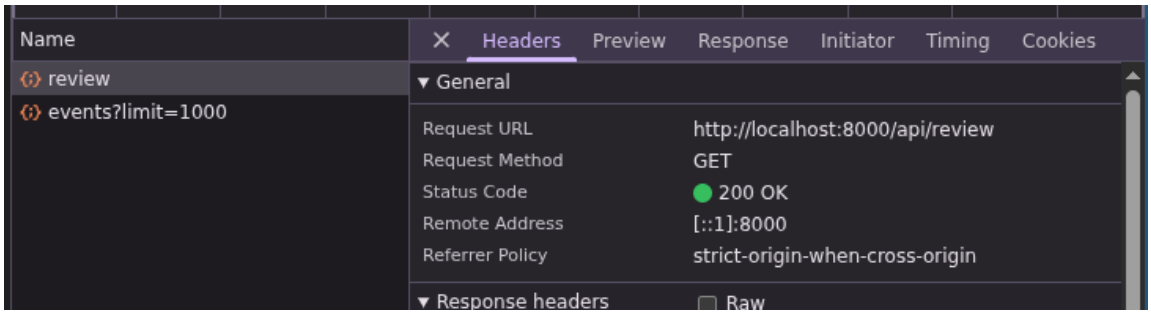
Topics covered:

- What is **frontend**
- What is **backend**
- What is an **API**

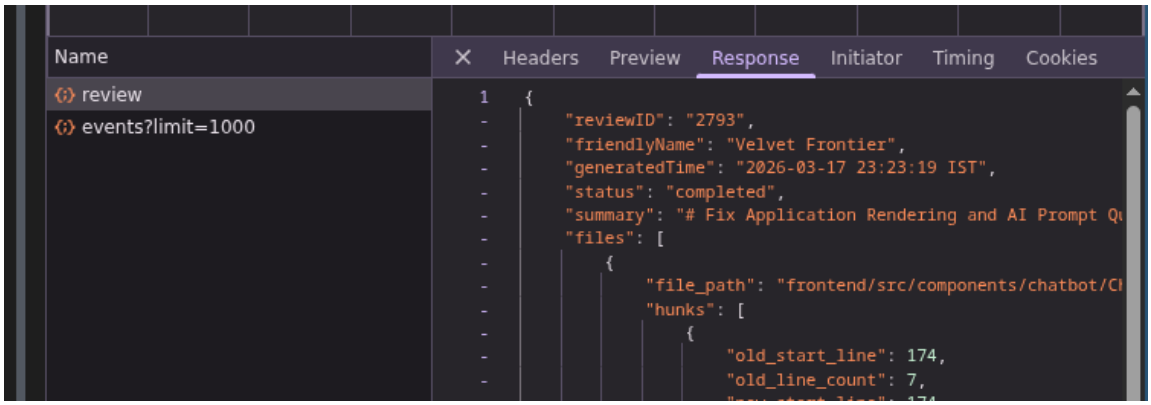
To make the concept practical, a live demonstration will be shown:

Example: git-lrc in local

- A **real API request** from the browser network tab



- The **server logs**
- The **API response returned to the browser**



5. Development: Building the AI Portfolio System (1hr 30minutes)

This is the main practical part of the workshop.

Students will transform their **resume into structured knowledge that an AI can understand**.

Step 1. (optional) Execute the prerequisites prompt in your editor

Hope you have already done so by following the prerequisites video

https://github.com/HexmosTech/ragfolio/blob/main/prompts/step_2_prerequisites.md

If any of the tools are not installed, please install the tool from the above links.

Step 2. Convert Resume into Knowledge Text M

Copy the content of your resume or downloaded LinkedIn resume.

Use the prompt below and get a structured resume from ChatGPT.

https://github.com/HexmosTech/ragfolio/blob/main/prompts/step_1_process-resume.md

Submit to ChatGPT and copy the output of the resume into a file called **resume.md** in the cloned repository.

Step 3. Creating Embeddings L

RAG:
https://docs.google.com/presentation/d/1NKxIObtFylsnSRN1g9uhbhnBaSm0z50WOo7IqkuxJZM/edit?slide=id.g3c5c5f36408_0_3356#slide=id.g3c5c5f36408_0_3356

Vector embeddings:
https://docs.google.com/presentation/d/1NKxIObtFylsnSRN1g9uhbhnBaSm0z50WOo7IqkuxJZM/edit?slide=id.g3c5c5f36408_0_3268#slide=id.g3c5c5f36408_0_3268

Vector database:
https://docs.google.com/presentation/d/1NKxIObtFylsnSRN1g9uhbhnBaSm0z50WOo7IqkuxJZM/edit?slide=id.g3c5c5f36408_0_3441#slide=id.g3c5c5f36408_0_3441

Students will run the project setup commands.

```
cd ragfolio/rag
uv sync
uv run create-embeddings.py
```

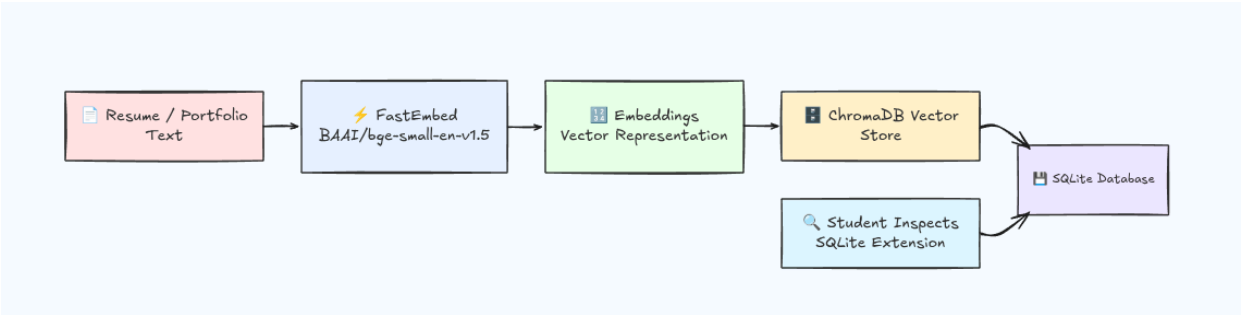
This step converts the **resume text into embeddings**.

AI models cannot efficiently search raw text.
Instead, the text is passed through an **embedding model (BAAI/bge-small-en-v1.5 via FastEmbed)** which converts each piece of text into a **vector — a list of numbers that represents the semantic meaning of the text**.

Texts with **similar meanings produce vectors that are close to each other in vector space**, allowing the system to find relevant information during search.

These embeddings are stored using **ChromaDB**, which internally uses an **SQLite database**.

Students can inspect the database using an **[SQLite extension]** to verify that the embeddings were successfully created.



Step 4. Building the Backend API M

Students will then implement an API that can be called from the frontend chatbot.

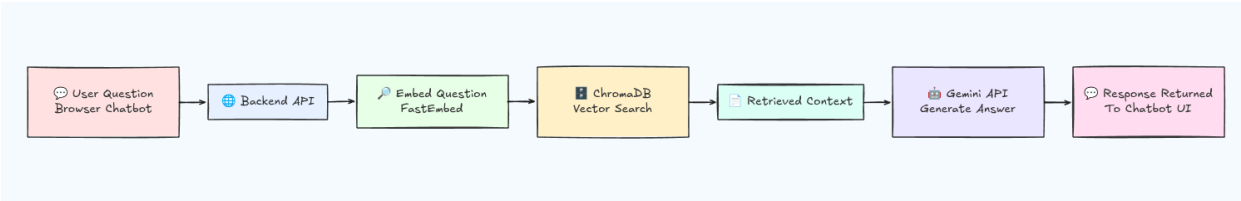
Prompt engineering:
https://docs.google.com/presentation/d/1NKxIObtFyIsnSRN1g9uhbhnBaSm0z50WOo7IqkuxJZM/edit?slide=id.g3c5c5f36408_0_3476#slide=id.g3c5c5f36408_0_3476

https://github.com/HexmosTech/ragfolio/blob/main/prompts/step_4_backend-controller.md

The chatbot will:

- accept user questions
- Send the question to the backend API
- retrieve relevant information from ChromaDB
- generate answers using the language model, which is the Gemini API

This step connects **frontend, backend, and AI systems together**.



Students will see how:

- The browser sends a request

- The backend processes the query
- The vector database retrieves relevant data
- The AI model generates a response

This demonstrates the **complete flow of a modern AI application**.

Check from the console

Step 4a. Validate Backend API

```
cd backend
uv sync
```

Start the backend server (from the backend directory):

```
uv run uvicorn main:app --reload --port 8000
```

Health check (should return {"status": "ok"}):

Mac/Linux

```
curl http://localhost:8000/api/health
```

Windows (Command Prompt)

```
Invoke-WebRequest http://localhost:8000/api/health
```

Test the /api/ask endpoint (replace "Your question here" as needed):

Mac/Linux

```
curl -X POST http://localhost:8000/api/ask \
-H "Content-Type: application/json" \
-d '{"question": "Why should I hire this person"}'
```

Windows (Command Prompt)

```
Invoke-RestMethod -Uri http://localhost:8000/api/ask `
-Method POST `
-Headers @{ "Content-Type" = "application/json" } `
-Body '{"question": "Why should I hire this person"}'
```

Step 5. Integrate the API with the UI L

https://github.com/HexmosTech/ragfolio/blob/main/prompts/step_5_frontend-api-integration.md

Step 5b. Validate Frontend

```
cd frontend
npm install
npm run dev
```

Open your browser and go to:

http://localhost:5000

Once the app loads:

- Open the chatbot UI
- Type: **“Why should I hire this person?”**
- Send the message

You should see:

- Your message appears instantly
- A loading state while the backend processes the request
- A response generated from the backend (`/api/ask` endpoint)

If everything is working correctly, the chatbot will return a relevant answer from the preloaded data.

Step 6. Improve the Site with Your Real Data L

Students can enhance the portfolio by replacing the sample content with their **own real information**.

This includes adding:

- Your **actual resume details**

- **Projects you have worked on**
- **Skills and technologies**
- Links to **GitHub, LinkedIn, or personal work**

You can also improve the **visual design and layout** of the site based on your personal taste.

This step is **optional and can also be done after deployment**, since UI customization and design tweaks often take more time.

While customizing the portfolio, make sure to **retain the chatbot at the beginning of the portfolio template**, so visitors can immediately interact with your AI assistant.

https://github.com/HexmosTech/ragfolio/blob/main/prompts/step_6_build-ui.md

6. Automated AI Code Review: Use git-lrc and review your code (20 minutes) M

During development, students will also learn the importance of **verifying AI-generated code**.

Although AI tools can generate code quickly, they can sometimes introduce:

- bugs
- security issues
- inefficient logic
- unnecessary complexity

To address this, the workshop introduces **git-lrc**, an AI-powered code review system.

Students will run a review on their generated code before committing it.

They will see:

- inline review comments
- detected issues
- Suggestions for improvement

This reinforces an important principle:

> **AI should assist development, but code should always be reviewed and verified.**

Students will then commit their code using Git.

Example workflow:

Install; https://github.com/HexmosTech/git-lrc?tab=readme-ov-file#install

```
git add .
git lrc review
```

- A review UI will open
- git-lrc reviews your code automatically
- If any issues are flagged, copy the suggestions and fix them using your AI assistant (e.g., Copilot)
- Stage the changes again:

```
git add .
git lrc review
```

- The code will be reviewed again
- Once critical issues are resolved, proceed with the **Commit & Push button in the UI**

https://github.com/HexmosTech/git-lrc

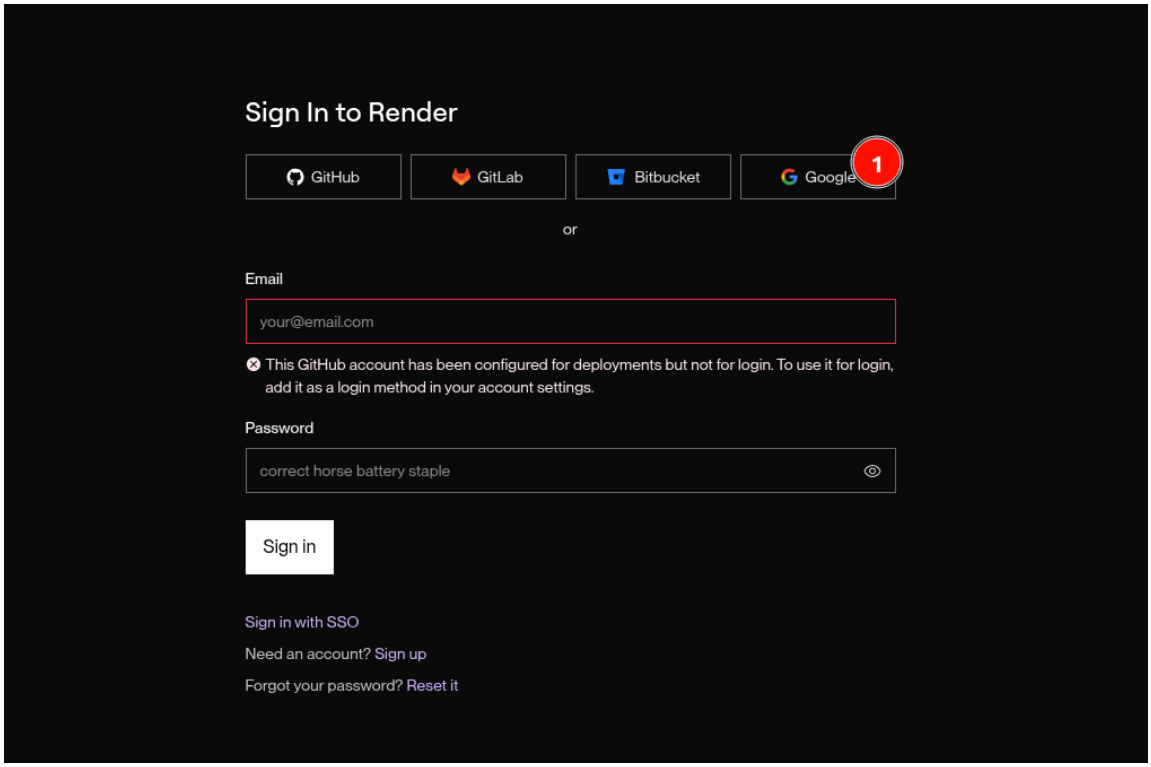
We are also looking for contributors, so look at our present project, **star it**, and use it every day.

7. Deployment: Deploying the Application (10 minutes) L

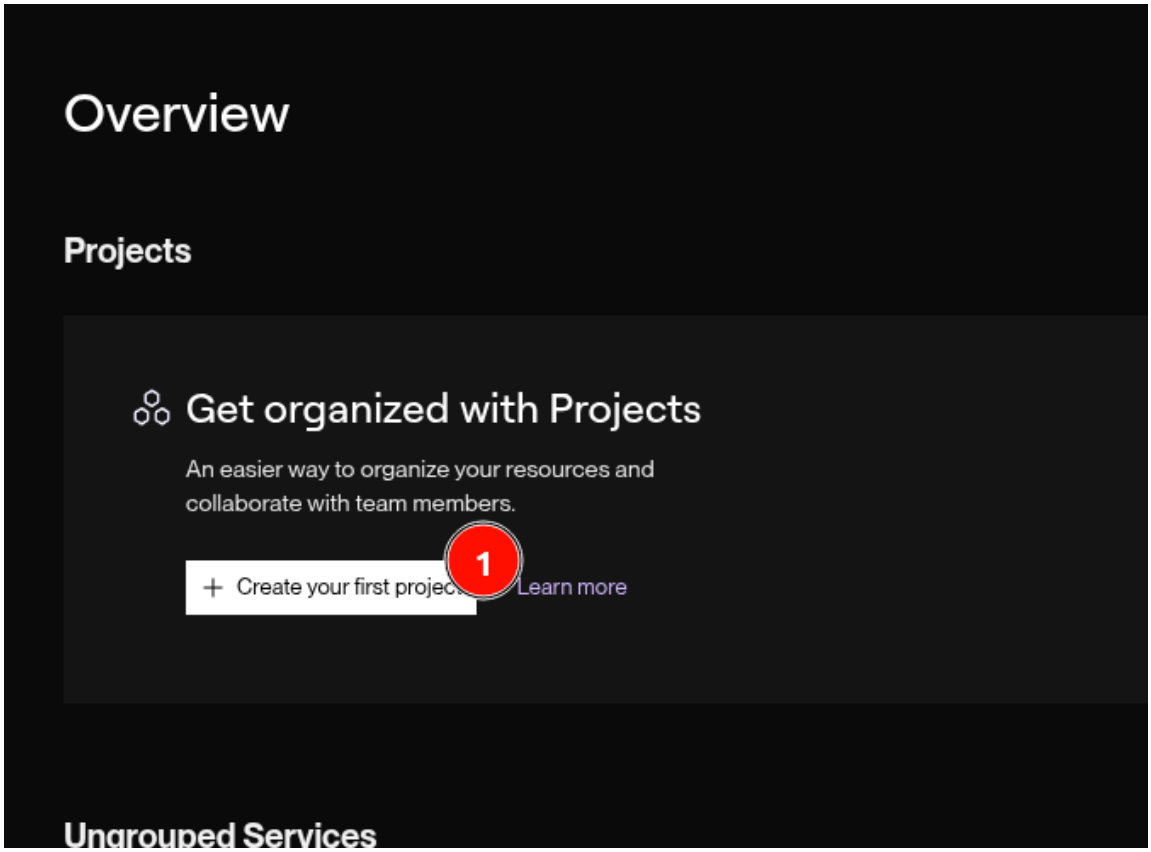
In the final stage of the workshop, students will deploy their project so that it becomes publicly accessible.

Deployment steps:

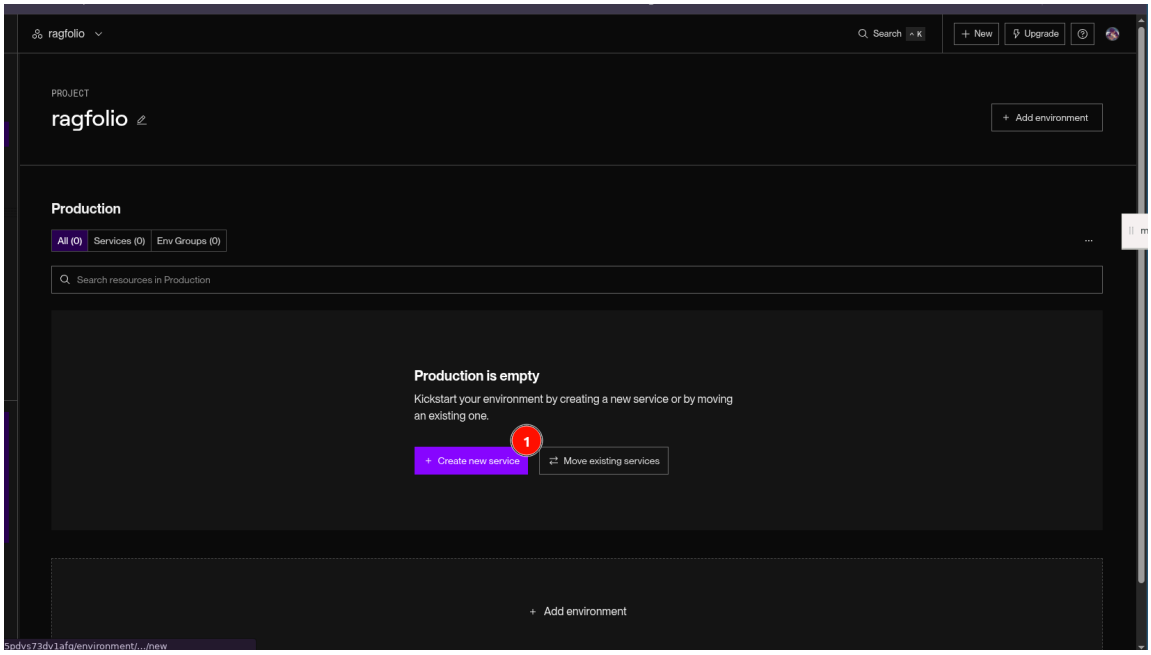
1. Log in to **Render**



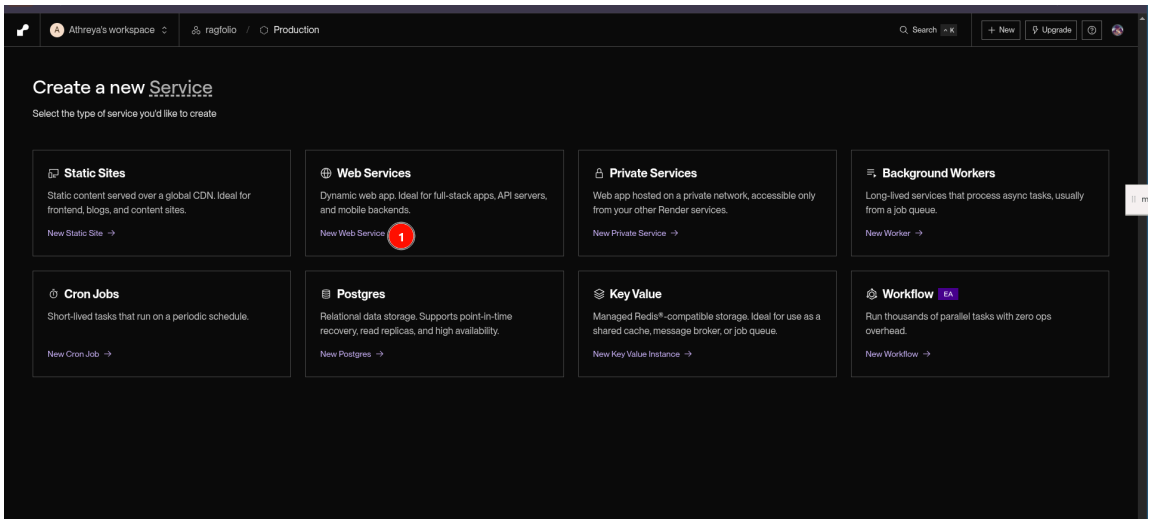
2. Create first project



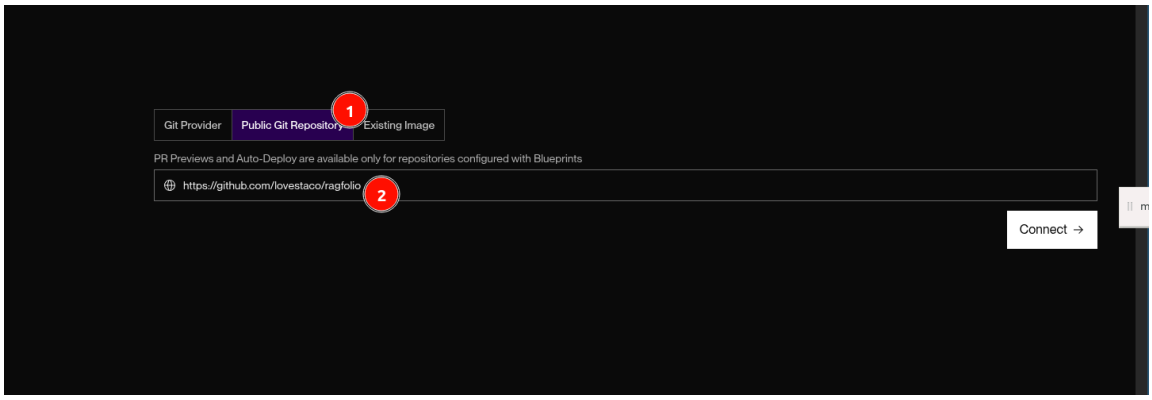
3. Create a new service



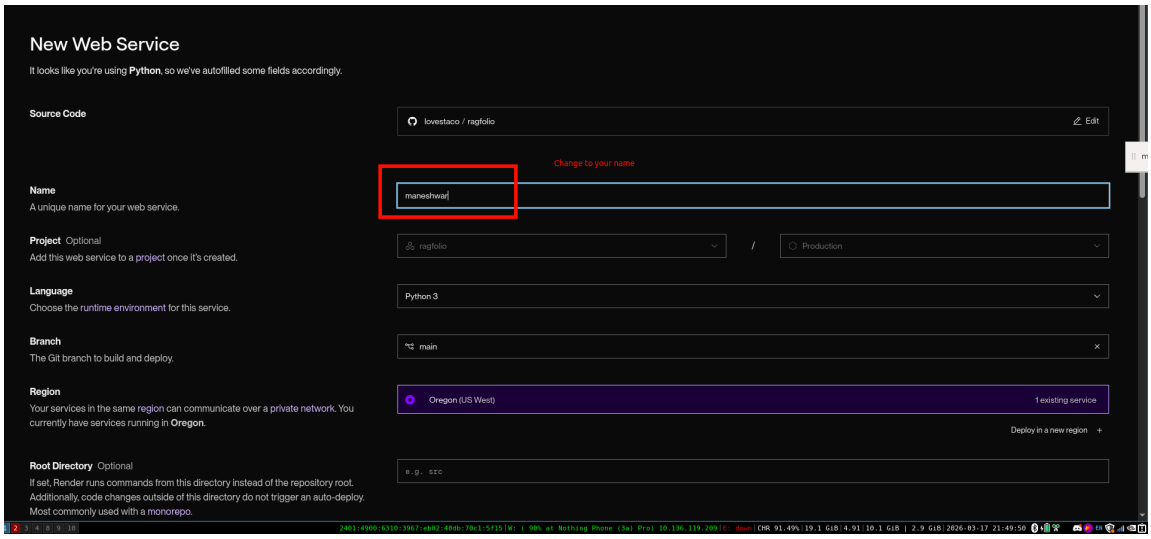
4. Select the “ web services” option



5. Paste your forked GitHub repository's public URL

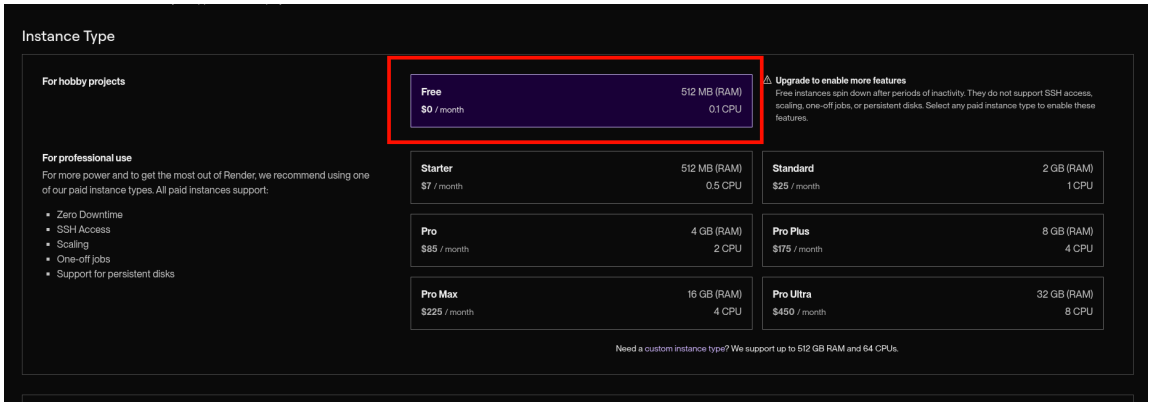


6. Change the name

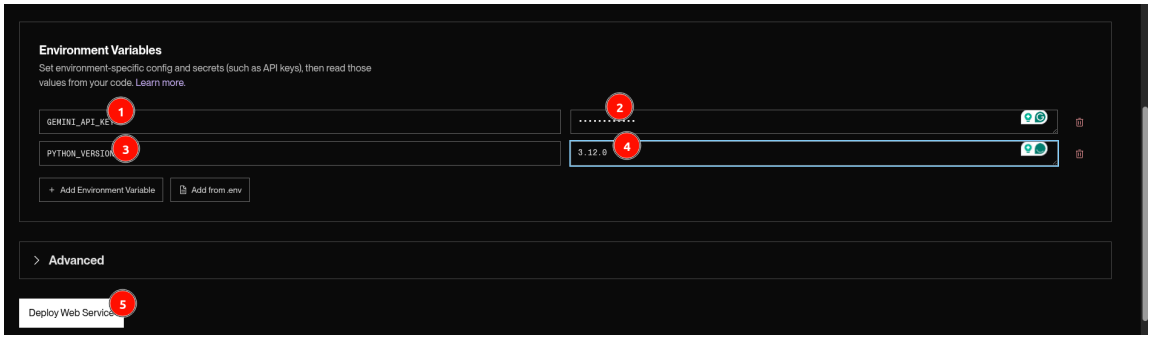


7. Add this configuration
    a. Build command to `./render-build.sh`
    b. Start command to `python -m uvicorn backend.main:app --host 0.0.0.0 --port $PORT`

8. Select FREE plan



9. Configure environment variables
    a. GEMINI_API_KEY: Your actual key
    b. PYTHON_VERSION: 3.12.0



10. Click Create Web Service

Within a few minutes, each student will receive a **live portfolio website with an AI chatbot**.

Example outcome:

https://tejas.onrender.com

Visitors can then interact with the portfolio by asking questions about the student's experience.

Our Recent Project (star us)
https://github.com/HexmosTech/git-lrc

—-

Students will also see a **Gemini API call from the terminal** to understand how external AI services are integrated into applications.

https://aistudio.google.com/api-keys

Windows

In the below command change replace the word with your API key `REPLACE_THIS_TEXT_WITH_ACTUAL_AISTUDIO_KEY` from AI studio

```
(Invoke-RestMethod `
-Uri
"https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent" `
-Method Post `
-Headers @{ "Content-Type"="application/json";
"X-goog-api-key"="REPLACE_THIS_TEXT_WITH_ACTUAL_AISTUDIO_KEY" } `
-Body (@{
contents = @(
@{
```

```
parts = @(
@{ text = "Explain what is an api request via curl in 2 sentences, I am
using gemini api key and making this curl, how is it working" }
)
}
)
} | ConvertTo-Json -Depth 5)
).candidates[0].content.parts[0].text
```

This step shows how an API can be executed from a powershell terminal.

Linux/ MacOS

```
curl
"https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-lates
t:generateContent" \
  -H 'Content-Type: application/json' \
  -H 'X-goog-api-key: REPLACE_THIS_TEXT_WITH_ACTUAL_AISTUDIO_KEY' \
  -X POST \
  -d '{
    "contents": [
      {
        "parts": [
          {
            "text": "Explain api request via curl in 2 sentences"
          }
        ]
      }
    ]
  }'
```